

There is one main ambiguity in the problem statement:

****Total volume vs. Number of servings:**** The statement "18 total cups of this drink" could be interpreted in two ways:

* 18 cups of the combined volume

* 18 individual servings

Let us assume the interpretation "18 cups of the combined volume".

Solution assuming 18 cups of combined volume:

****Step 1: Find the proportion of lemonade in one drink:****

- Lemonade: $1 \frac{1}{4}$ cups

- Iced tea: $\frac{1}{4}$ cup

- Total: $1 \frac{1}{4} + \frac{1}{4} = 1 \frac{1}{2}$ cups

- Lemonade proportion: $(1 \frac{1}{4}) / (1 \frac{1}{2}) = \frac{5}{6}$

****Step 2: Calculate the amount of lemonade in the pitcher:****

- Total volume: 18 cups

- Lemonade proportion: $\frac{5}{6}$

- Volume of lemonade: $18 * (\frac{5}{6}) = 15$ cups

Therefore, there are 15 cups of lemonade in the pitcher.

The answer is 15.

Q:

A deep-sea monster rises from the waters once every hundred years to feast on a ship and sate its hunger. Over three hundred years, it has consumed 847 people. Ships have been built larger over time, so each new ship has twice as many people as the last ship. How many people were on the ship the monster ate in the first hundred years?

A:

Let us solve it using algebra. Let x be the number of people on the ship the monster ate in the first hundred years.

The number of people on the ship eaten in the second hundred years is $2x$, and in the third hundred years is $4x$.

Therefore, the total number of people eaten over three hundred years is $x + 2x + 4x = 847$.

Combining like terms, we get $7x = 847$.

Dividing both sides by 7, we find $x = 121$.

Therefore, there were 121 people on the ship the monster ate in the first hundred years.

The answer is 121.

Q:

[{problem}](#)

A:

D.2 MBPP

You are an expert Python programmer. Solve the following Python programming problems.

Q:

Write a function to find the similar elements from the given two tuple lists.
Your code should pass these tests:

```
assert similar_elements((3, 4, 5, 6),(5, 7, 4, 10)) == (4, 5)
assert similar_elements((1, 2, 3, 4),(5, 4, 3, 7)) == (3, 4)
assert similar_elements((11, 12, 14, 13),(17, 15, 14, 13)) == (13, 14)
```

A:

```
def similar_elements(test_tup1, test_tup2):
    res = tuple(set(test_tup1) & set(test_tup2))
    return (res)
```

Q:

Write a python function to identify non-prime numbers.
Your code should pass these tests:

```
assert is_not_prime(2) == False
assert is_not_prime(10) == True
assert is_not_prime(35) == True
```

A:

```
import math
def is_not_prime(n):
    result = False
    for i in range(2,int(math.sqrt(n)) + 1):
        if n % i == 0:
            result = True
    return result
```

Q:

Write a function to find the largest integers from a given list of numbers using heap queue algorithm.
Your code should pass these tests:

```
assert heap_queue_largest([25, 35, 22, 85, 14, 65, 75, 22, 58],3)==[85, 75, 65]
assert heap_queue_largest([25, 35, 22, 85, 14, 65, 75, 22, 58],2)==[85, 75]
assert heap_queue_largest([25, 35, 22, 85, 14, 65, 75, 22, 58],5)==[85, 75, 65, 58, 35]
```

A:

```
import heapq as hq
def heap_queue_largest(nums,n):
    largest_nums = hq.nlargest(n, nums)
    return largest_nums
```

Q:

{problem}

Your code should past these tests:

{tests}

A:

E Additional Discussion

Weighting Examples Theorem 5 requires $\alpha > 0$ and $\beta > 0$, and it is easy to see that both assumptions are necessary. If $\alpha = 0$ then each training set $\mathcal{D}_t$ contains no exogenously labeled data, and if $\beta = 0$ then all of the exogenously labeled data could be incorrect. In either case, and given a worst-case initial LLM g_0 that returns an incorrect response to every prompt, the learner has no way to determine the correct response to any prompt.

The astute reader will note that α can also be set arbitrarily close to zero in Theorem 5 without weakening the convergence guarantee. This curious property is a consequence of an idealized assumption that we made for the sake of analytical tractability. Specifically, we assumed that the learner can match the conditional response distribution of every prompt in the training data, no matter how infrequently the prompt appears in the data (see Definition 1). In practice, constraints on training time and model size will prevent a learner from perfectly fitting the training data. So it would be useful to extend our results to account for the possibility of an imperfect learner, and we expect that any such extension would imply a non-zero lower bound on α . Nonetheless, our current results tell us something interesting – computational limitations are the *only* barrier to learning an arbitrarily good LLM, and not, as one might expect, the quality of the weak labeler.

Filtering Non-Synthetic Data Algorithm 2 has the property that it only applies filtering on LLM-generated data. As discussed, this accurately models existing methods in the literature.

However, if we consider applying the quality function q on data produced by the weak labeler (that is, data that is not LLM-generated), then there is an alternate solution to the data generation problem. It can be shown that $O(\log(1/\varepsilon)/\beta)$ invocations of the weak labeler would suffice to correctly label all but ε fraction of the prompts in P , and such a dataset could be given to a strong learner to produce an LLM that achieves $O(\varepsilon)$ error. It is worthwhile to reason about why such a solution cannot be deployed in practice.

First, and most crucially – it has been repeatedly demonstrated empirically in the literature that training directly on 100% correct human responses is sub-optimal compared to training on self-generated synthetic data [Singh et al., 2024, Zelikman et al., 2022]. Indeed this is confirmed in our own experiments where we find that *finetuning on correct answers (Gold SFT) is dramatically outperformed by recursive training*, hence motivating our theoretical study of the problem.